Project Factory - Speedy
Data de Publicação: 28/04/2026

Grupo:
Nycolas Souza - 20230989
Luan Ribeiro - 20230692
Lohanne Guedes - 20220085
Kira Sousa - 20231205

1. Introdução

O Speedy é um protótipo de condução autónoma desenvolvido para navegação reativa em ambiente competitivo. O projeto foca-se na fusão sensorial entre visão computacional (YOLO), para o reconhecimento de sinalética e limites de pista, e na utilização de LiDAR para a deteção dinâmica de aberturas (gaps) e desvio de obstáculos em tempo real. Através de uma arquitetura baseada em ROS 2 Jazzy, o sistema orquestra decisões de alto nível e controlo de hardware de baixa latência, visando a otimização de trajetórias e a superação de obstáculos físicos como rampas e superfícies irregulares.

2.1. Requisitos Funcionais

Código	Requisito	Descrição
RF01	Condução Remota	O sistema deve permitir que o usuário controle o veículo manualmente através de um comando.
RF02	Desvio de Obstáculos	O veículo deve ser capaz de identificar e evitar colisões com objetos durante o movimento.
RF03	Percepção Visual	O sistema deve identificar elementos da pista e objetos ao redor através de uma câmera.
RF04	Seleção de Operação	O usuário deve conseguir alternar entre o controle manual e modo automático de forma simples.
RF05	Visualização de Status	O sistema deve disponibilizar dados de desempenho e sensores para monitoramento externo.
RF06	Frenagem de Segurança	O veículo deve parar imediatamente caso perca a conexão com o sistema de controle central.
RF07	Monitoramento de Movimento	O sistema deve medir a orientação e aceleração do veículo para registro de telemetria.
RF08	Mapeamento e Localização	O veículo deve ser capaz de criar mapas do ambiente e identificar a sua posição neles em tempo real.

2.2. Requisitos Não-Funcionais

Código	Requisito	Descrição
RNF01	Framework ROS 2	Toda a comunicação e lógica de nós deve ser gerida pelo ROS 2 Jazzy.
RNF02	Automação Ansible	A instalação de dependências e replicação do ambiente deve ser garantida pelo Ansible.
RNF03	Firmware micro-ROS	O controle de baixo nível (motores e sensores) deve rodar no ESP32 com micro-ROS.
RNF04	Mapeamento via LiDAR	A criação de mapas e o desvio de objetos devem utilizar um sensor LiDAR.
RNF05	Identificação via YOLO	O reconhecimento visual de objetos e sinais deve ser processado por modelos YOLO.
RNF06	Odometria e Navegação	O veículo deve estimar a sua pose e velocidade em tempo real, fundindo dados de navegação inercial (IMU) e odometria (sensores Hall nas rodas/motor).
RNF07	Ambiente Distrobox	O desenvolvimento deve ser feito em containers Distrobox para garantir que o software rode igual em qualquer PC.
RNF08	Desligamento Seguro	O sistema deve garantir o encerramento controlado do Raspberry Pi para evitar a corrupção do cartão Micro-SD

3. Diagrama de Componentes

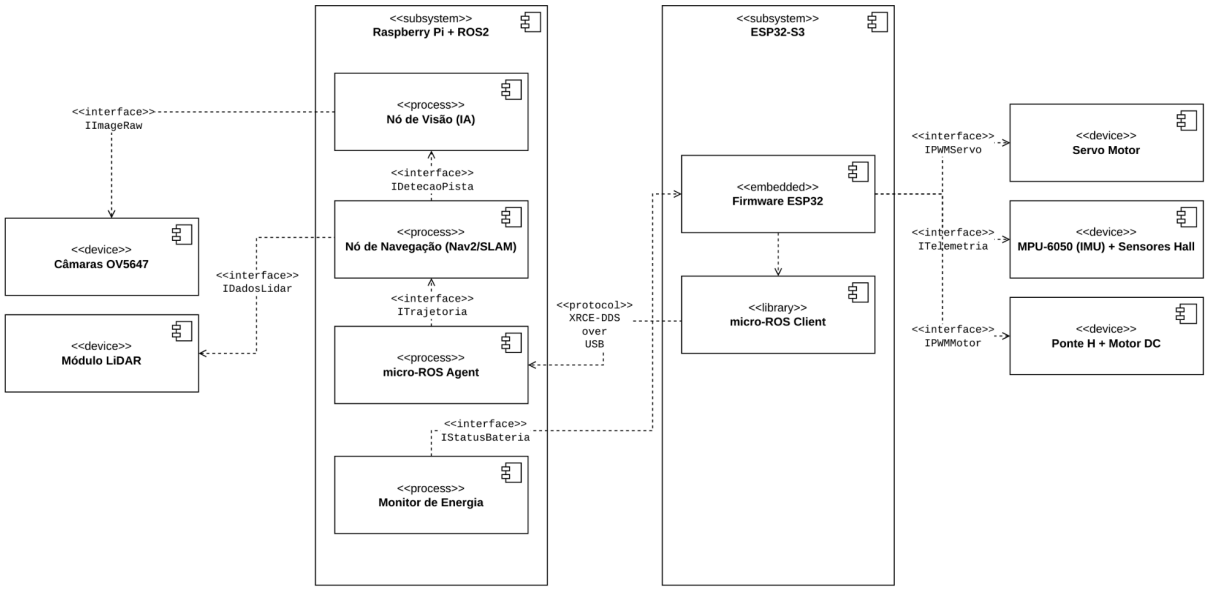


Diagrama de Componentes

O diagrama ilustra a arquitetura distribuída do veículo, composta pelo processamento de alto nível no Raspberry Pi (ROS2) e pelo controlo em tempo real no ESP32-S3. Os dados brutos dos periféricos de perceção (LiDAR e Câmaras) são processados pelos nós de Visão e Navegação para o cálculo da trajetória ideal, cujas decisões são transmitidas via protocolo XRCE-DDS ao micro-ROS no ESP32. O firmware traduz estes comandos em sinais PWM para o Motor DC e Servo Motor, enquanto monitoriza a integridade do hardware através dos dados inerciais do MPU-6050 e da odometria precisa gerada pelos Sensores Hall. Em paralelo, o ESP32 vigia a tensão elétrica e reporta estados críticos ao Monitor de Energia no Raspberry Pi, permitindo que este execute um encerramento controlado (poweroff) para evitar a corrupção de dados no cartão SD em caso de baixa tensão ou desconexão da bateria.

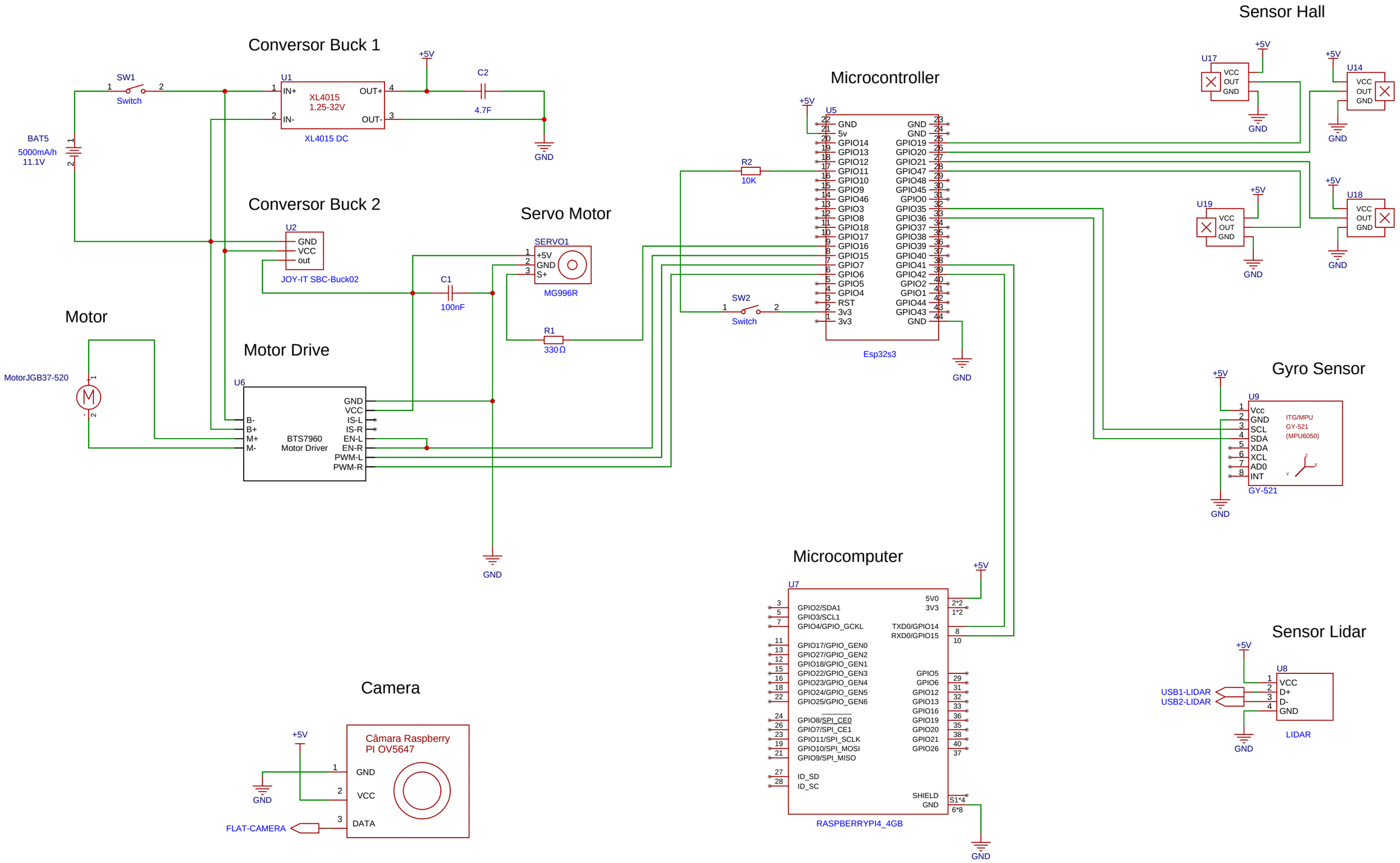
4. Lista Simplificada de Materiais

Descrição	**Quantidade**
Raspberry Pi 5 4GB	1
Microcontrolador ESP32-S3	1
Motor JGB37-520 12V 600RPM	1
Servo Motor MG996R 180°	1
Controlador de Motor Ponte H BTS7960	1
Sensor LIDAR LDROBOT D500	1
Câmara para Raspberry PI OV5647 160°	1
Sensor Giroscópio/Acelerómetro MPU-6050	1
Sensores Hall	3
Bateria LiPo 3S 11.1V 5000mAh	1

(Lista completa disponível na secção de anexos)

5. Anexos

1. Código Fonte: https://github.com/nycocado/speedy
2. Diagrama Elétrico: https://github.com/nycocado/speedy/tree/main/media/milestone-1/circuit.pdf
3. Bill of Materials: https://github.com/nycocado/speedy/tree/main/media/milestone-1/bom.xlsx


Conversor Buck 1
U1
XL4015
1.25-32V
IN+
IN-
OUT+
OUT-
XL4015 DC
+5V
C2
4.7F
GND
SW1
Switch
BAT5
5000mA/h
11.1V
Conversor Buck 2
U2
GND
VCC
out
JOY-IT SBC-Buck02
Servo Motor
SERVO1
+5V
GND
S+
MG996R
C1
100nF
R1
330Ω
Motor
MotorJGB37-520
Motor Drive
U6
B-
B+
M+
M-
BTS7960
Motor Driver
GND
VCC
IS-L
IS-R
EN-L
EN-R
PWM-L
PWM-R
Microcontroller
U5
R2
10K
SW2
Switch
Esp32s3
Sensor Hall
U17
U14
U19
U18
VCC
OUT
GND
Gyro Sensor
U9
Vcc
GND
SCL
SDA
XDA
XCL
AD0
INT
ITG/MPU
GY-521
(MPU6050)
GY-521
Microcomputer
U7
GPIO2/SDA1
GPIO3/SCL1
GPIO4/GPIO_GCKL
GPIO17/GPIO_GEN0
GPIO27/GPIO_GEN2
GPIO18/GPIO_GEN1
GPIO22/GPIO_GEN3
GPIO23/GPIO_GEN4
GPIO24/GPIO_GEN5
GPIO25/GPIO_GEN6
GPIO8/SPI_CE0
GPIO7/SPI_CE1
GPIO11/SPI_SCLK
GPIO10/SPI_MOSI
GPIO9/SPI_MISO
ID_SD
ID_SC
5V0
3V3
TXD0/GPIO14
RXD0/GPIO15
GPIO5
GPIO6
GPIO12
GPIO13
GPIO16
GPIO19
GPIO20
GPIO21
GPIO26
SHIELD
RASPBERRYPI4_4GB
Sensor Lidar
U8
VCC
D+
D-
GND
USB1-LIDAR
USB2-LIDAR
LIDAR
Camera
Câmara Raspberry
PI OV5647
FLAT-CAMERA